

Information Extraction with Azure AI Document Intelligence

What This Chapter Covers

- Pre-built models — invoices, receipts, ID cards, business cards, contracts
 - Custom model training on your own domain-specific document datasets
- Human-in-the-loop feedback mechanism for continuous model improvement
- Dev Container setup with VS Code for a reproducible training environment
 - Azure infrastructure provisioning via automated PowerShell scripts
- Complete Jupyter Notebook walkthrough — cell-by-cell code explanation
 - Model versioning, data upload, layout analysis, and OCR pipeline
- On-premises deployment via Docker containers for air-gapped environments
 - Production architecture, security, cost management, and best practices

SECTION 1 — THEORY AND CONCEPTUAL FOUNDATION

1. Introduction — The Document Intelligence Problem

Organisations across every industry are drowning in documents. Banks process millions of loan applications. Insurance companies handle thousands of claims per day. Healthcare systems manage oceans of patient records, referral forms, and insurance cards. Logistics companies process endless invoices, delivery notes, and customs declarations. Governments digitise archives containing decades of paper records.

The challenge is not just volume — it is variety. Every organisation uses slightly different document layouts, field names, and formats. An invoice from one supplier looks nothing like an invoice from another. A customs declaration in Brazil has different fields from one in Germany. Traditional rule-based extraction systems (templated parsers, regular expressions) break the moment a document layout changes, requiring expensive manual reconfiguration.

Azure AI Document Intelligence solves this problem using machine learning. Instead of defining rigid rules about where a field appears on a page, it learns the semantic meaning of document content — understanding that '£1,250.00' near the word 'Total' is an amount, regardless of whether it appears in the top-right corner or the bottom-centre of the page. This semantic, layout-aware understanding makes it robust to the inevitable variation in real-world documents.

The Core Value Proposition Azure AI Document Intelligence transforms the manual, error-prone, expensive process of document data entry into an automated, scalable, API-driven pipeline. What previously required a team of data entry operators processing 100 documents per day can be automated to process 10,000 documents per hour.

2. What Is Azure AI Document Intelligence?

Azure AI Document Intelligence (formerly Azure Form Recognizer) is a machine learning-based cloud service that automates data extraction from documents. It combines several AI capabilities into a unified document understanding pipeline:

- Optical Character Recognition (OCR) — converts the visual text in an image or PDF into machine-readable Unicode characters, handling printed text, handwriting, mixed scripts, and varied fonts
- Layout Analysis — understands the structural organisation of a document: headings, paragraphs, tables (including merged cells and spanning headers), checkboxes, selection marks, and page regions
- Key-Value Pair Extraction — identifies labelled fields and their values across the document, even when field labels and values are spatially separated or formatted differently from document to document

- **Table Extraction** — reconstructs tabular data with accurate row-column relationships, even for complex tables with merged cells, nested headers, or irregular layouts
- **Entity Recognition** — identifies and classifies named entities such as dates, amounts, addresses, person names, and organisation names within their document context

The service is accessible through three interfaces — each appropriate for different use cases:

Interface	Best Used For
Azure AI Document Intelligence Studio	Visual testing, model configuration, labelling custom training data, and exploring API results without writing code
Python / .NET / Java SDK	Programmatic integration in application code, batch processing pipelines, and automation workflows
REST API	Language-agnostic integration, microservices architectures, and scenarios where SDK support is not available

3. Pre-built Models vs. Custom Models — When to Use Each

Document Intelligence offers two fundamentally different approaches to extraction, and choosing the right one is the first architectural decision you make when building a document processing pipeline.

3.1 Pre-built Models

Pre-built models are Microsoft-trained models that can extract common fields from standard document types immediately, without any training data or configuration on your part. They have been trained on millions of real-world documents and understand the semantic layout of their target document types across many languages, geographies, and visual styles.

Pre-built Model	Fields Extracted
Invoices	Vendor name/address, customer name/address, invoice ID, invoice date, due date, purchase order number, subtotal, tax, total, line items (description, quantity, unit price, amount)
Receipts	Merchant name/address/phone, transaction date/time, line items (name, quantity, price), subtotal, tax, tip, total, payment method
Identity Documents	First/last name, date of birth, document number, expiry date, sex, nationality, address — for passports, driver's licences, and national IDs
Business Cards	Contact name, job title, company name, email addresses, phone numbers, addresses, websites

Bank Statements	Account details, transaction history, opening/closing balances, transaction dates and amounts
Health Insurance Cards	Member name, member ID, group number, payer ID, plan details, employer information
US Tax Forms (W-2, 1040)	Employee/employer information, wages, tax withheld, federal and state tax fields
Mortgage Documents	Borrower information, loan details, property information, financial figures
General Documents	Generic key-value pairs, tables, and entities from any document type without pre-defined fields
Layout	Full structural analysis — paragraphs, tables, selection marks, page structure — without field extraction

3.2 Custom Models

Custom models are trained on your organisation's own document samples to extract fields specific to your domain, your templates, and your business processes. They are the right choice when:

- Your documents are proprietary forms not covered by any pre-built model
- Your document layouts differ significantly from the generic versions the pre-built models were trained on
- You need to extract highly domain-specific fields (internal order codes, custom reference numbers, proprietary classification fields)
- Your extraction accuracy requirements are higher than what the general pre-built model achieves on your specific documents

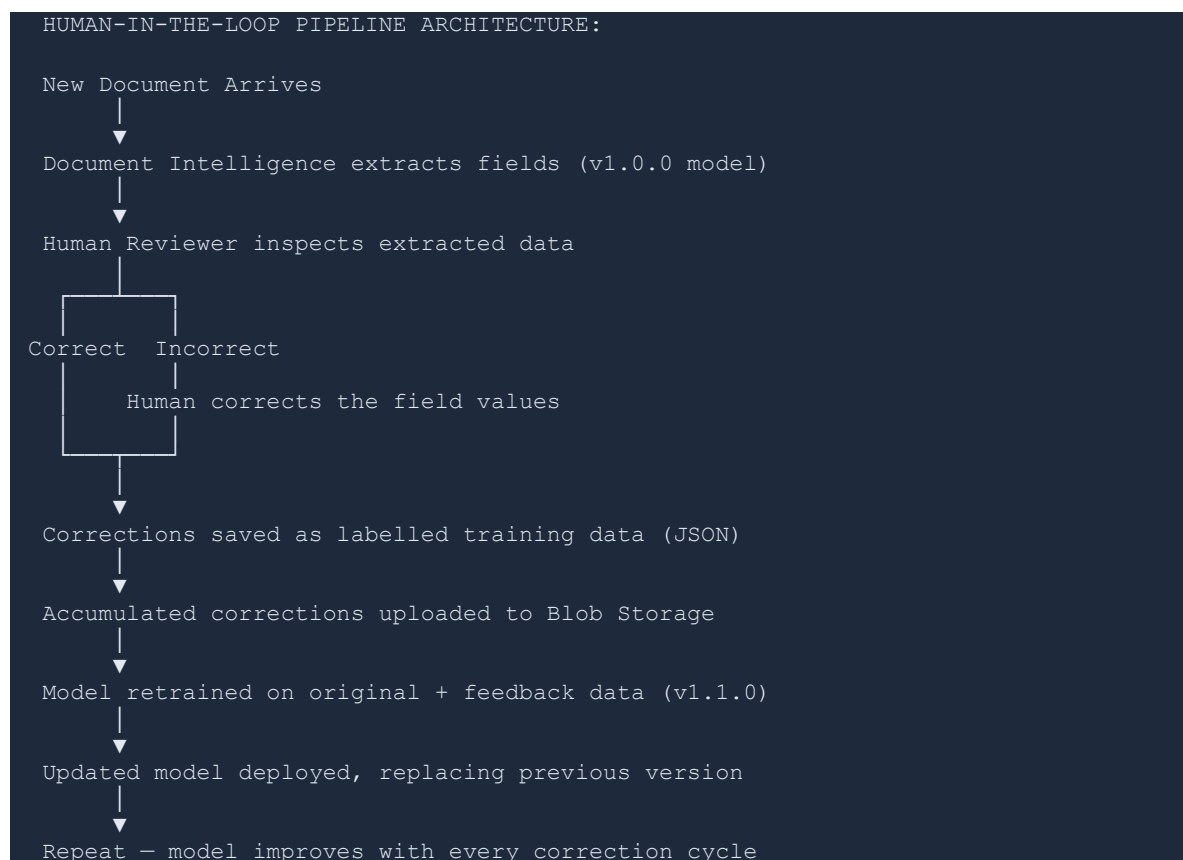
There are three types of custom models:

Custom Model Type	How It Works & When to Use It
Custom Template Model	Trained on documents with consistent, fixed layouts (e.g., your company's standard purchase order form). Works best when field positions are predictable. Requires as few as 5 training samples.
Custom Neural Model	Trained on documents with variable layouts (e.g., invoices from many different suppliers). Uses deep learning to understand field meaning regardless of position. Requires 10-50+ training samples for good accuracy.
Composed Model	Combines multiple custom models into a single endpoint. The service automatically routes each document to the most appropriate sub-model. Ideal for processing multiple document types in a single pipeline.

4. The Human-in-the-Loop Feedback Architecture

No machine learning model achieves perfect accuracy immediately, especially on real-world documents with unusual layouts, poor scan quality, or edge cases not represented in the training data. The human-in-the-loop (HITL) feedback architecture addresses this by building a continuous improvement cycle into the document processing pipeline.

The core idea is straightforward: when the model extracts data from a document and a human reviewer corrects any mistakes, those corrections are valuable labelled training data. Instead of discarding corrections after they are applied, the HITL pipeline captures them, formats them as new training samples, and uses them to retrain the model — making it progressively more accurate on exactly the types of documents and errors your organisation encounters in practice.



Why This Matters A model trained on 50 generic invoice samples may achieve 80% field-level accuracy on your specific invoice formats. After 200 correction cycles feeding back into retraining, the same model architecture can achieve 95%+ accuracy — trained specifically on the idiosyncrasies of your suppliers' invoices. The model learns your data.

5. On-Premises Deployment via Docker Containers

While the cloud-hosted Azure AI Document Intelligence service is ideal for most organisations, certain scenarios require on-premises deployment — running the Document Intelligence processing engine within your own network, on your own hardware, without sending document data to Microsoft's cloud:

- Air-gapped environments — government, defence, or critical infrastructure systems with strict network isolation requirements
- Data sovereignty compliance — regulations requiring that specific document types (e.g., citizen data, classified materials) never leave a defined geographic or network boundary
- Ultra-low latency requirements — processing documents in real-time at the point of capture (factory floor scanners, retail POS systems) where cloud round-trip latency is unacceptable
- Intermittent connectivity — processing documents in field locations with unreliable internet connectivity

Microsoft provides Docker container images for Document Intelligence models that can be run on any compliant Docker host — whether on-premises servers, edge devices, or private cloud infrastructure. Critically, even when running on-premises, the container must maintain billing connectivity to Azure: it periodically reports usage metrics to an Azure Cognitive Services endpoint for billing reconciliation. The actual document data never needs to leave your network, but the usage telemetry does.

Container Type	Min CPU	Min Memory	Recommended Memory
Read (OCR)	8 cores	10 GB	24 GB
Layout Analysis	8 cores	16 GB	24 GB
Invoice	8 cores	16 GB	24 GB
Receipt	8 cores	11 GB	24 GB
Business Card	8 cores	16 GB	24 GB
General Document	8 cores	12 GB	24 GB
ID Document	8 cores	8 GB	24 GB
Custom Template	8 cores	16 GB	24 GB

Hardware Requirement Note All minimum and recommended values refer to Docker resource limits (set via `--cpus` and `--memory` flags), not the physical host machine specification. The host machine should have additional headroom beyond these limits for the OS and other processes. Each CPU core must run at ≥ 2.6 GHz.

6. Prerequisites and Development Environment

This chapter covers the complete workflow including custom model training within a Dev Container environment. The following prerequisites are required:

Prerequisite	Purpose & Notes
Azure Subscription	Required for all Azure resource provisioning — Document Intelligence, Blob Storage, and billing
Docker Desktop	Required for the Dev Container development environment and for on-premises container deployment. Version 20.10+ recommended.
Visual Studio Code	Primary IDE for the Jupyter notebook workflow. Must have the 'Dev Containers' extension (ms-vscode-remote.remote-containers) installed.
Python 3.9+	Runtime for the training notebooks. Managed automatically within the Dev Container.
Azure CLI	Used for authenticating to Azure from within the Dev Container (az login). Version 2.40+ recommended.
Git	For cloning the reference repository containing the training pipeline code and Dev Container configuration.

Reference GitHub repositories:

```
# Original implementation by James McRoft
https://github.com/jamesmcroft/document-intelligence-user-feedback-processor

# Fork the repository to your own GitHub account before making changes
# Then clone your fork locally:
git clone https://github.com/<your-username>/document-intelligence-user-feedback-processor
cd document-intelligence-user-feedback-processor
```

SECTION 2 — PRACTICAL IMPLEMENTATION — PORTAL, NOTEBOOKS & CONTAINERS

7. Step-by-Step: Creating the Document Intelligence Resource

Step 1 — Provision the Azure AI Services Resource

1. Log in to portal.azure.com and search for 'Azure AI services' in the top search bar.
2. Click '+ Create'. The creation form will appear — the multi-service Azure AI account provides Document Intelligence alongside Vision, Speech, and Language under a single endpoint and key, which simplifies management for projects using multiple AI services.
3. Fill in the required fields: Subscription, Resource Group (create new or select existing, e.g., 'azure-ai-ps'), Region (e.g., 'East US'), Name (e.g., 'azure-ai-13243'), and Pricing Tier (Standard S2 for production workloads, Free F0 for development).
4. Click 'Review + create', then 'Create'. Wait for the deployment notification — typically 60-90 seconds.
5. After deployment completes, navigate to the resource and open the 'Keys and Endpoint' blade. Note your endpoint URL and Key 1 — you will need these for SDK configuration.

Step 2 — Open Document Intelligence Studio

6. From the Azure Portal, search for 'Azure AI Studio' or navigate directly to studio.azureml.microsoft.com.
7. In Azure AI Studio, locate your hub in the left panel and open your project workspace.
8. In the left navigation, expand 'AI Services' then select 'Vision + Document'. This opens the Document Intelligence features panel.
9. You will see the Document Intelligence Studio interface with sections for: Prebuilt models, Custom models, and Document analysis tools.

Step 3 — Analysing a Receipt with the Pre-built Model

10. In Document Intelligence Studio, under 'Prebuilt', click on 'Receipts'. This opens the receipt model testing interface.
11. You will see the interface has three tabs at the top-right: Fields, Result, and Code. The Fields tab shows a structured list of all fields the model can extract. The Result tab shows raw JSON output. The Code tab generates ready-to-run SDK code.
12. To test the model, either: drag and drop a receipt image/PDF onto the upload area, click 'Browse for a file' to select from your local system, or use the 'Fetch from URL' option to provide a publicly accessible image URL.

13. Click 'Run analysis'. The 'Analyze options' dialog appears, allowing you to configure: Analysis range (current document or all documents), Page range (all pages or specific range), Optional detection (barcodes, language detection), and Premium detection (high resolution, style font, formula detection).
14. Click 'Run analysis' in the dialog. The model processes the document and populates the Fields panel with all extracted values — merchant name, transaction date, line items, subtotal, tax, and total.
15. Switch to the 'Code' tab to see the Python, C#, or REST code that performs exactly this extraction programmatically. This generated code includes your endpoint URL and model ID pre-populated, ready to copy into your application.

8. Setting Up the Dev Container Environment

The human feedback pipeline is developed and run within a VS Code Dev Container — a fully configured, reproducible development environment defined in code. The Dev Container specification in the repository (`.devcontainer/devcontainer.json`) defines exactly which tools, Python packages, and extensions are installed, ensuring that the training environment is identical regardless of your host machine's operating system or configuration.

Why Dev Containers?

- Reproducibility — everyone working on the project gets an identical environment, eliminating 'it works on my machine' problems
- Isolation — all dependencies are contained within the Docker container, with no risk of version conflicts with other projects on your host
- Portability — the environment runs identically on Windows, macOS, and Linux
- Onboarding speed — a new team member can have a fully configured development environment running with a single VS Code command

Step-by-Step Dev Container Setup

16. Clone the repository to your local machine and open the folder in VS Code.
17. Open the VS Code Command Palette using `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (macOS).
18. Type 'Dev Containers' in the Command Palette and select 'Dev Containers: Reopen in Container'. VS Code will detect the `.devcontainer` configuration and begin building the container image.
19. Wait for the container build to complete — the first build downloads the base Docker image and installs all dependencies, which may take 3-10 minutes depending on your internet speed. Subsequent opens are near-instant because Docker caches the layers.
20. Once the container is running, open a new terminal within VS Code (Terminal → New Terminal). Run the `hostname` command to verify you are inside the container:

```
# You should see a container-generated hostname (a hex string), NOT your host
machine's hostname
$ hostname
a61962c55d49 # example - your value will differ

# This confirms you are running inside the Dev Container
```

21. Verify the Azure CLI is available and authenticate to your Azure account:

```
$ az --version # should show Azure CLI 2.x.x

# Log in using your tenant ID (find this in Azure Portal → Microsoft Entra ID)
$ az login --tenant <your-tenant-id>

# A browser window will open for authentication
# After successful login, you will see your subscription details printed
```

22. Set file permissions to allow the training scripts to create new files within the repository directory:

```
$ sudo chmod -R 777 ../document-intelligence-user-feedback-processor/

# -R : apply recursively to all files and subdirectories
# 777: read, write, and execute permissions for owner, group, and others
# This is necessary because the training scripts generate new OCR output files
# and JSON label files within the project directory
```

9. Automated Azure Infrastructure Provisioning

The repository includes a PowerShell setup script (Setup-Environment.ps1) that automates the provisioning of all required Azure resources. Rather than manually creating each resource through the Portal, this script creates everything needed for the training pipeline in a single command.

Resources Created by the Setup Script

Azure Resource	Purpose in the Pipeline
Resource Group	Logical container for all pipeline resources, enabling unified management and cost tracking
Azure AI Document Intelligence	The Document Intelligence service instance providing the extraction and training APIs
Azure Blob Storage Account	Stores training documents (PDFs), label files (JSON), OCR output files, and model feedback data
Storage Container: training-data	Blob container holding the PDF training documents uploaded for model training
Azure Log Analytics Workspace	Centralised logging for pipeline operations, model training events, and error tracking
Azure Monitor / Application Insights	Performance monitoring and telemetry for the feedback processing workflow

Running the Setup Script

23. Open a new PowerShell terminal within the Dev Container (Terminal → New Terminal, then switch to PowerShell using the terminal profile selector).

24. Run the setup script with your desired configuration parameters:

```
PS /workspaces/document-intelligence-user-feedback-processor> \
  ./Setup-Environment.ps1 \
  -DeploymentName dacintel-feedback-loop \
  -Location centralus \
  -SkipInfrastructure $true

# Parameter explanations:
# -DeploymentName : prefix used for naming all created Azure resources
# -Location      : Azure region for all resources (use the same region
#                as your Document Intelligence resource for lowest
latency)
# -SkipInfrastructure : $true skips resource creation if already provisioned;
#                    set to $false on first run
```

25. The script prints progress messages as it works: 'Starting environment setup...', 'Deploying infrastructure...', 'Starting infrastructure deployment...'. Full provisioning typically takes 3-8 minutes.

26. After the script completes, navigate to portal.azure.com → Resource groups and search for your deployment name prefix (e.g., 'rg-dacintel-feedback-loop'). You will see all provisioned resources listed, confirming successful deployment.

Infrastructure as Code Principle The automated setup script embodies the Infrastructure as Code (IaC) principle — your entire Azure environment is defined and reproducible from a version-controlled script. Anyone on your team can recreate the exact same environment by running one command, and you can destroy and recreate it cleanly at any time.

10. Jupyter Notebook Walkthrough — Custom Model Training Pipeline

The training pipeline is implemented as a Jupyter notebook, enabling an interactive, step-by-step execution flow where you can inspect intermediate results and make decisions at each stage. The following walkthrough explains every cell in the notebook in comprehensive detail.

Cell 1 — Importing All Modules and Libraries

```
import os
from dotenv import dotenv_values
from azure.identity import DefaultAzureCredential
from modules.app_settings import AppSettings
from modules.model_training_client import ModelTrainingClient
from modules.document_canvas import DocumentCanvas
from modules.document_intelligence_label import DocumentIntelligenceLabel
from modules.document_intelligence_result_formatter import \
    DocumentIntelligenceResultFormatter
```

What each import provides:

- `import os` — provides access to the operating system interface, specifically `os.path` for constructing file paths in a cross-platform way. Used throughout the notebook to build paths to PDF files, label files, and output directories.
- `from dotenv import dotenv_values` — reads a `.env` file and returns its contents as a dictionary. This allows the notebook to load configuration (endpoint URLs, storage account names, model IDs) from a configuration file rather than hardcoding them. The `.env` file is excluded from version control, preventing accidental exposure of credentials.
- `from azure.identity import DefaultAzureCredential` — imports Microsoft's chain-of-credentials authentication class. In a Dev Container authenticated via Azure CLI (`az login`), `DefaultAzureCredential` automatically uses the CLI's cached token. The same code works in production on Azure services using Managed Identity, without any code changes.
- `from modules.app_settings import AppSettings` — a custom class in the repository that wraps the configuration dictionary from `dotenv`, providing typed, named access to settings (e.g., `settings.document_intelligence_endpoint` instead of `config['DOCUMENT_INTELLIGENCE_ENDPOINT']`). This improves code readability and provides IDE autocomplete.
- `from modules.model_training_client import ModelTrainingClient` — a custom client class that wraps the Azure AI Document Intelligence SDK, providing high-level methods (`upload_training_data`, `create_model`, `run_layout_analysis`) that encapsulate the low-level API details.
- `from modules.document_canvas import DocumentCanvas` — a custom class for rendering document pages with labelled field overlays in a Jupyter notebook environment. Used to visually display which fields the model has identified and where they are located on the page.
- `from modules.document_intelligence_label import DocumentIntelligenceLabel` — represents a single labelled field on a document, including its name, value, and bounding box coordinates. Used to construct the feedback data structure.
- `from modules.document_intelligence_result_formatter import DocumentIntelligenceResultFormatter` — converts the model's extraction results and human feedback into the JSON format expected by the Document Intelligence training API (the `.labels.json` format).

Cell 2 — Configuring Authentication with Selective Credential Chain

```
working_dir = os.path.abspath('')
settings = AppSettings(dotenv_values(f'{working_dir}/config.env'))

azure_credential = DefaultAzureCredential(
    exclude_environment_credential=True,
    exclude_managed_identity_credential=True,
    exclude_shared_token_cache_credential=True,
    exclude_interactive_browser_credential=True,
    exclude_powershell_credential=True,
    exclude_visual_studio_code_credential=False, # ← ENABLED
    exclude_cli_credential=False                 # ← ENABLED
```

```
)
```

Understanding the credential exclusion pattern in depth:

DefaultAzureCredential is a 'chain' credential — it tries multiple authentication methods in a defined order until one succeeds. The full chain includes: environment variables, managed identity, shared token cache, Visual Studio Code credentials, Azure CLI credentials, interactive browser, and PowerShell. In a Dev Container environment, this chain needs to be restricted for two important reasons:

- **Security:** The container may inadvertently pick up stale credentials from shared token caches or environment variables that do not belong to the target Azure subscription. Explicitly disabling these prevents authentication confusion.
- **Determinism:** By allowing only VS Code and CLI credentials, the authentication source is predictable and auditable — you know exactly which identity is being used.

`exclude_environment_credential=True` — disables reading credentials from environment variables (AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, etc.). These are typically used in production CI/CD pipelines but should not be set in the Dev Container to avoid confusion with service principal credentials.

`exclude_managed_identity_credential=True` — disables Managed Identity authentication. Managed Identity is available when running on Azure compute (VMs, App Service, AKS) but not in a local Dev Container.

`exclude_visual_studio_code_credential=False` (enabled) — allows use of the VS Code Azure Account extension's cached token, which is set when you sign in through the VS Code Azure extension.

`exclude_cli_credential=False` (enabled) — allows use of the Azure CLI's cached token from `az login --tenant`. This is the primary authentication method used in the Dev Container workflow.

`os.path.abspath("")` returns the absolute path of the current working directory — the notebook's location. This is used to construct the path to `config.env`, which must be in the same directory as the notebook.

Cell 3 — Defining Model Identity: Name, Version, and ID

```
# The semantic name for this model - matches the document type it analyses
model_name = 'invoices'

# Semantic versioning for the initial release
initial_model_version = '1.0.0'

# The fully qualified model ID registered in Azure AI Document Intelligence
# Format: '{model_name}-{version}' - e.g., 'invoices-1.0.0'
initial_model_id = f'{model_name}-{initial_model_version}'
```

Model naming and versioning conventions are critical for production model management:

- `model_name = 'invoices'` — a semantic name that describes what the model processes. In a production system with multiple document types, you might have models named 'purchase_orders', 'supplier_invoices', 'customs_declarations', etc.
- `initial_model_version = '1.0.0'` — semantic versioning (MAJOR.MINOR.PATCH) communicates the nature of each change: MAJOR for breaking changes to the extraction schema, MINOR for accuracy improvements from new training data, PATCH for minor bug fixes. When you retrain with human feedback, you increment to 1.1.0.
- `initial_model_id = f'{model_name}-{initial_model_version}'` — the unique identifier used to reference this specific model version in the Document Intelligence API. This ID is what you pass when calling the extraction API to specify exactly which model version should process your document.

Cell 4 — Creating the Model Training Client

```
model_training_client = ModelTrainingClient(  
    settings=settings,  
    use_azure_credential=False,  
    azure_credential=azure_credential  
)
```

The `ModelTrainingClient` constructor accepts three parameters:

- `settings=settings` — the `AppSettings` object loaded from `config.env`. This provides the client with all necessary configuration: Document Intelligence endpoint URL, storage account name and key, container names, and other service-specific settings.
- `use_azure_credential=False` — this parameter controls whether the client should attempt to create its own `DefaultAzureCredential` instance internally. Setting it to `False` tells the client: 'Do not create your own credential chain — I will provide one explicitly.'
- `azure_credential=azure_credential` — passes the carefully configured `DefaultAzureCredential` object created in Cell 2. By passing it explicitly, you ensure the restricted credential chain (VS Code + CLI only) is used for all API calls made by this client, maintaining the security and determinism guarantees established earlier.

This pattern of externalising credential creation and injecting it into service clients is a form of Dependency Injection — it makes the authentication strategy explicit, configurable, and testable.

Cell 5 — Uploading Training Data and Creating the Initial Model

```
# Clear any residual training data from previous runs  
# This prevents stale documents from contaminating the new training set  
model_training_client.delete_training_data('Invoice_6')  
  
# Upload the initial training document set from the local model_training  
directory  
# This copies all PDF files to Azure Blob Storage's training-data container  
model_training_client.upload_training_data(f'{working_dir}/model_training')
```

```
# Initiate model training on the uploaded data
# Azure Document Intelligence begins analysing the documents and learning field
locations
invoice_model = model_training_client.create_model(model_name=initial_model_id)
```

Each method in this cell triggers a distinct phase of the training pipeline:

- `delete_training_data('Invoice_6')` — removes a specific document ('Invoice_6') from the Blob Storage training container before uploading the fresh training set. This is important when iterating: if Invoice_6 was a mislabelled document from a previous run, keeping it would pollute the new training run with incorrect labels. The argument is the blob name (document filename) to remove.
- `upload_training_data(path)` — recursively copies all PDF files from the specified local directory to the Azure Blob Storage training-data container. After this call, the Azure Portal's Storage Browser will show the uploaded PDFs (e.g., invoice_1.pdf, invoice_2.pdf ... invoice_n.pdf) in the training-data blob container, ready for model training.
- `create_model(model_name=initial_model_id)` — submits the model training job to Azure AI Document Intelligence. The service: reads all documents from the Blob Storage container, performs layout analysis on each, identifies field patterns across all documents, and trains a custom neural model to extract those fields from new documents. Training typically takes 2-10 minutes depending on the number and complexity of training documents.

Cell 6-8 — Layout Analysis and OCR File Generation

```
# Define paths for a feedback document (a new invoice to be reviewed)
pdf_path = os.path.join(pdf_dir, 'invoice_feedback.pdf')
pdf_feedback_path = os.path.join(pdf_dir, 'invoice_feedback.ocr.json')

# Run layout analysis using the initial model
# This generates a detailed OCR + layout JSON file for the feedback document
model_training_client.run_layout_analysis(
    pdf_path,          # input: the new document to analyse
    pdf_feedback_path, # output: where to save the OCR JSON file
    initial_model_id   # which model version to use for analysis
)
```

`run_layout_analysis()` performs two operations simultaneously on the feedback document:

- OCR analysis — extracts all text from the PDF, producing word-level bounding boxes with their associated text content. This forms the foundation that human reviewers work on top of — they correct which words are assigned to which fields, rather than typing field values from scratch.
- Layout analysis — identifies the document's structural elements: paragraphs, tables, page regions. This spatial context helps the labelling interface accurately render field bounding boxes in alignment with the document's visual layout.

The output is a `.ocr.json` file — a structured JSON document containing the complete text extraction result with full spatial coordinates. This file is required by the `DocumentCanvas` class in the next cell to render the document for human review.

Cells 9-11 — Displaying the Document for Human Review

```
# Create a DocumentCanvas object for this invoice
doc_canvas = DocumentCanvas(working_dir)

# Load the PDF and its associated OCR output and field definitions
# Returns a list of canvas objects – one per page
c canvases = doc_canvas.load_pdf(
    pdf_path, # the original PDF file
    document_fields_path, # JSON defining which fields this model extracts
    pdf_feedback_path # the OCR JSON generated in the previous cell
)

# Render each page in the Jupyter notebook output
for canvas in canvases:
    display(canvas)
```

DocumentCanvas is the interactive review layer of the feedback pipeline:

- `doc_canvas.load_pdf()` combines three data sources: the original PDF (for visual reference), the field definitions (so the interface knows which fields to show), and the OCR JSON (so it can display text content and bounding boxes). It returns one canvas object per document page.
- `display(canvas)` renders the page inside the Jupyter notebook as an interactive widget. The rendered view shows the document page with colour-coded bounding boxes drawn around each field that the model identified, labelled with the field name and extracted value.
- Human reviewers can then visually inspect the rendered page and compare the model's extraction (shown in the bounding box overlays) against the actual document content. Any discrepancy — a wrong field value, a missed field, or an incorrectly bounded region — is noted for correction in the next cell.

Cells 12-13 — Capturing Human Feedback and Rendering Labels

```
# Retrieve all labelled regions from the document canvas
# doc_canvas.get_document_labels() returns the model's extraction result
# as a list of label regions (bounding boxes with field names and values)
labels = [
    DocumentIntelligenceLabel(label_region, doc_canvas.fields)
    for label_region in doc_canvas.get_document_labels()
]

# Render each label's visual representation in the notebook
# This shows bounding boxes drawn around each identified field
for label in labels:
    display(label.render())
```

This cell captures the model's extraction results as structured label objects:

- `doc_canvas.get_document_labels()` retrieves the list of identified field regions from the canvas. Each region represents a field the model found — its name (e.g., 'InvoiceTotal'), its extracted text value, and the polygon coordinates of the bounding region on the page.
- `DocumentIntelligenceLabel(label_region, doc_canvas.fields)` — wraps each raw label region in a typed object that knows the full schema of expected fields (from `doc_canvas.fields`). This wrapper provides the `render()` method and handles the JSON serialisation needed later.
- `label.render()` — produces a visual notebook widget for each label: a small image showing the cropped section of the document page within the label's bounding box, annotated with the field name and extracted value. This allows the reviewer to see exactly which portion of the document produced each extracted value, making it straightforward to spot errors.

At this stage, the human reviewer examines each rendered label and can either accept it (the extraction is correct) or flag it for correction. In a more complete implementation, this would be an interactive widget where the reviewer can directly edit the extracted value.

Cell 14 — Saving Corrected Labels as Training JSON

```
# Serialise the labels (with any human corrections applied) to JSON
# This creates the .labels.json file required by Document Intelligence for
training
DocumentIntelligenceResultFormatter.save_to_labels_json(
    labels,          # the list of DocumentIntelligenceLabel objects
    pdf_file_name,   # filename of the document these labels belong to
    pdf_labels_path  # output path for the labels JSON file
)
```

`save_to_labels_json()` converts the label list into the specific JSON schema required by Azure AI Document Intelligence for training. The output format is a `.labels.json` file — a structured document that maps each field name to its bounding box coordinates and text value within a specific document. For example:

```
# Example .labels.json structure
{
  "document": "invoice_feedback.pdf",
  "labels": [
    {
      "label": "InvoiceId",
      "value": [
        {
          "page": 1,
          "text": "INV-2024-00142",
          "boundingRegions": [
            { "page": 1, "polygon": [100, 50, 380, 50, 380, 75, 100, 75] }
          ]
        }
      ]
    },
    // ... additional labelled fields
  ]
}
```

These `.labels.json` files are what transform your document collection from unlabelled raw data into supervised training data. Each file tells the model: 'In this specific document, the field `InvoiceId` has value `INV-2024-00142` and it appears at these coordinates.' Accumulate enough of these labelled

examples and the model learns to generalise — finding InvoiceId in new documents it has never seen, regardless of exactly where on the page it appears.

Cell 15 — Retraining with Updated Model Version

```
# Define the updated model version — minor bump from 1.0.0 to 1.1.0
# indicating improved accuracy from user feedback, no schema changes
updated_model_version = '1.1.0'

# Construct the new model ID following the same naming convention
updated_model_id = f'{model_name}-{updated_model_version}'

# Upload the feedback documents (with corrected labels) to Blob Storage
# pdf_dir contains both the original training PDFs and the new feedback documents
model_training_client.upload_training_data(pdf_dir)

# Initiate training of the updated model on the combined dataset
updated_model = model_training_client.create_model(
    model_name=updated_model_id
)
```

This cell implements the retraining step of the human feedback loop:

- `updated_model_version = '1.1.0'` — the minor version increment (1.0.0 → 1.1.0) correctly signals that this is an improvement to the same model (same extraction schema, same field names) rather than a breaking change. If you added new fields to extract, that would be a major version bump to 2.0.0.
- `model_training_client.upload_training_data(pdf_dir)` — uploads the combined training dataset: the original labelled invoices from the initial training run, plus the new feedback documents with their human-corrected label files. The model will be trained on all of this data together.
- `model_training_client.create_model(model_name=updated_model_id)` — triggers a new training run. Azure AI Document Intelligence reads all documents and their associated .labels.json files from the Blob Storage container and trains a new model version. The existing 1.0.0 model is not replaced — both versions exist in your Document Intelligence resource, allowing you to A/B test them or roll back.

Cells 16-17 — Validating the Updated Model

```
# Construct the output path for the updated model's analysis result
# Naming convention includes the model version for traceability
df_updated_analysis_path = os.path.join(
    pdf_dir,
    f'{pdf_file_name}.ocr.{updated_model_version}.json'
)

# Run layout analysis using the updated model on the same feedback document
# This lets us directly compare v1.0.0 and v1.1.0 extraction results
model_training_client.run_layout_analysis(
    pdf_path,
    df_updated_analysis_path,
```

```
        updated_model_id          # note: using the NEW model version
    )

    # Display the updated extraction results in the notebook
    # Visually compare against the v1.0.0 results shown earlier
    canvases_updated = doc_canvas.load_pdf(
        pdf_path, document_fields_path, df_updated_analysis_path
    )
    for canvas in canvases_updated:
        display(canvas)
```

This validation step closes the feedback loop by answering the critical question: did retraining actually improve accuracy? By running the same feedback document through both the 1.0.0 model and the 1.1.0 model and displaying both results side-by-side in the notebook, you can visually verify that the fields the reviewer had to correct in version 1.0.0 are now extracted correctly by version 1.1.0. This before-and-after comparison is essential for building confidence in the retraining process before deploying the updated model to production.

11. Deploying Document Intelligence On-Premises with Docker

When cloud deployment is not possible, Docker containers allow the Document Intelligence service to run entirely within your own network. The following walkthrough covers the complete on-premises deployment of the Read (OCR) container.

Step 1 — Gather Required Configuration from Azure

Even when running on-premises, the container requires connectivity to Azure for billing. Retrieve these values from your Azure AI Services resource:

27. In the Azure Portal, navigate to your Azure AI Services resource.
28. Open the 'Keys and Endpoint' blade. Copy the Endpoint URL (e.g., <https://di-wotjgmeslmnj4.cognitiveservices.azure.com/>) and Key 1. These will be passed to the container as environment variables.

Step 2 — Run the Docker Container

```
docker run -d \
    --name DocIntelLocal \
    -e EULA=accept \
    -e billing=https://di-wotjgmeslmnj4.cognitiveservices.azure.com/ \
    -e apiKey=dc6f81687ff745e5adbc7faef906af1a \
    -e AzureCognitiveServiceLayoutHost=http://DocIntelLocal:5000 \
    -p 5000:5000 \
    mcr.microsoft.com/azure-cognitive-services/form-recognizer/read-3.1
```

Every flag in this command serves a specific purpose:

- `docker run -d` — starts the container in detached mode (runs in the background as a daemon process). Without `-d`, the container output streams to your terminal and the container stops when you close the terminal.
- `--name DocIntelLocal` — assigns a human-readable name to the running container. Without this, Docker assigns a random name. The name is used to reference the container in subsequent `docker stop`, `docker logs`, and `docker exec` commands.
- `-e EULA=accept` — sets the EULA environment variable to 'accept', which is required by Microsoft's container licensing terms. The container will refuse to start without this flag, printing an error instructing you to accept the End User License Agreement.
- `-e billing=https://di-wotjgmeslmnj4.cognitiveservices.azure.com/` — the endpoint URL of your Azure Cognitive Services resource. The container sends periodic usage telemetry to this endpoint so Microsoft can accurately bill your subscription. Replace this with your actual endpoint from the Azure Portal.
- `-e apiKey=dc6f81687ff745e5adbc7faef906af1a` — the API key for your Azure resource, used to authenticate the billing telemetry reports. Replace with your actual key. Never commit real API keys to source control.
- `-e AzureCognitiveServiceLayoutHost=http://DocIntelLocal:5000` — configures the Read container's internal service discovery. It tells the Read OCR component where to find the Layout analysis service. When both components run as the same container (as in this single-container deployment), this points back to the container's own internal port.
- `-p 5000:5000` — port mapping in the format `host_port:container_port`. This makes the container's internal port 5000 accessible as port 5000 on the host machine. After the container starts, your applications call `http://localhost:5000/formrecognizer/v2.1/read/analyze` (the same API signature as the cloud service) to perform OCR locally.
- `mcr.microsoft.com/azure-cognitive-services/form-recognizer/read-3.1` — the fully qualified Docker image name from Microsoft Container Registry (MCR). Docker will pull this image on first run (~2-5 GB download depending on model size). Subsequent runs use the cached local image.

Step 3 — Verify the Container is Running

```
# Check that the container started successfully
docker ps --filter name=DocIntelLocal

# Check container logs for startup messages
docker logs DocIntelLocal --tail 50

# Test the service health endpoint
curl http://localhost:5000/
# Expected: HTML page with service status and API documentation link

# Test a local OCR request (same API as cloud service)
curl -X POST http://localhost:5000/formrecognizer/v2.1/read/analyze \
  -H 'Content-Type: application/json' \
  -d '{"url":"https://example.com/sample_invoice.pdf"}'
```

Step 4 — Pointing Your Application at the Local Container

The on-premises container exposes the identical REST API as the cloud service. The only change required in your application code is the endpoint URL:

```
# Cloud deployment endpoint
CLOUD_ENDPOINT = 'https://di-wotjgmeslmnj4.cognitiveservices.azure.com/'

# On-premises container endpoint
LOCAL_ENDPOINT = 'http://localhost:5000'

# The SDK client accepts either endpoint — no other code changes needed
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.core.credentials import AzureKeyCredential

# Switch between cloud and on-premises by changing only the endpoint
client = DocumentIntelligenceClient(
    endpoint=LOCAL_ENDPOINT,          # or CLOUD_ENDPOINT
    credential=AzureKeyCredential('any-value-for-local') # not validated locally
)
```

This API-level compatibility is a deliberate design principle: the same application code works unchanged whether pointing at the cloud service or the on-premises container. This allows you to develop and test using the cloud service, then deploy to an on-premises container for production without any application code modifications.

12. Production Architecture and Best Practices

Concern	Recommended Approach
Model Versioning	Always use semantic versioning (MAJOR.MINOR.PATCH) and never overwrite model IDs. Keep all versions deployed simultaneously to enable A/B testing and instant rollback.
Training Data Quality	More labelled examples is generally better, but quality matters more than quantity. 50 accurately labelled invoices outperform 500 poorly labelled ones. Validate all training labels before uploading.
Feedback Volume Threshold	Accumulate at least 10-20 new feedback documents before triggering a retraining cycle. Retraining on 1-2 corrections at a time produces noisy improvements and wastes compute.
Pre-built vs Custom	Always evaluate the pre-built model first. If it achieves acceptable accuracy (>85% field-level) on your documents, use it — custom model training adds operational overhead. Only invest in custom training when pre-built accuracy is insufficient.
Container Sizing	For production on-premises deployment, provision 2-3x the minimum memory specification. Document Intelligence models load large neural network weights into memory, and running near the minimum limit causes performance degradation under concurrent load.

Billing Connectivity	On-premises containers must have reliable outbound HTTPS connectivity to your Azure endpoint for billing telemetry. If billing connectivity fails for more than 24 hours, the container stops processing requests. Plan your network policy accordingly.
Secret Management	API keys for on-premises containers should be stored in a secrets manager (HashiCorp Vault, Azure Key Vault via private endpoint) rather than hardcoded in docker run commands or Compose files.

13. Interview Questions and Answers

Q: What is the difference between a pre-built model and a custom model in Azure AI Document Intelligence?

A: Pre-built models are Microsoft-trained, immediately usable models for common document types (invoices, receipts, IDs). They require no training data and work out-of-the-box with high accuracy on standard document formats. Custom models are trained on your organisation's own labelled document samples to handle proprietary forms, domain-specific layouts, or document types not covered by pre-built models. The trade-off is: pre-built models have zero training overhead but less flexibility; custom models require labelled training data and operational overhead but can achieve higher accuracy on your specific documents. Always start with a pre-built model evaluation before investing in custom training.

Q: Why is human-in-the-loop retraining important, and how does it improve model accuracy over time?

A: No model achieves perfect accuracy on all real-world documents, especially on edge cases, unusual layouts, or low-quality scans not well represented in the training data. Human-in-the-loop retraining captures extraction errors as they occur in production, converts the human corrections into labelled training data, and periodically retrains the model on the expanded dataset. This creates a virtuous cycle: the model encounters a document type it handles poorly → humans correct the mistakes → corrections become training data → the retrained model handles that document type better. Over months of production operation, a custom model trained this way continuously improves on exactly the types of documents and errors it actually encounters.

Q: What is a Dev Container and why is it used for the training pipeline?

A: A Dev Container is a Docker-based development environment defined in a `.devcontainer/devcontainer.json` file within the project repository. It specifies the exact OS, tools, libraries, VS Code extensions, and configuration needed to work on the project. When a developer opens the project in VS Code with the Dev Containers extension, VS Code automatically builds and starts the container and opens the workspace inside it. This ensures every team member works in an identical environment, eliminating environment-specific bugs and making onboarding trivial. For a training pipeline, it also ensures that the Python package versions, Azure CLI version, and authentication configuration are consistent — preventing hard-to-diagnose failures caused by subtle environment differences.

Q: Why does the on-premises Docker container still require connectivity to Azure, even though it runs locally?

A: The on-premises container requires outbound HTTPS connectivity to an Azure Cognitive Services billing endpoint for usage telemetry — it reports how many API calls it has processed so Microsoft can bill your subscription accurately. The actual document data never leaves your network; only anonymised usage counters are sent. If billing connectivity fails for an extended period (typically 24 hours), the container stops accepting API requests as a safeguard against unbilled usage. This billing-connected-but-data-local architecture is a deliberate design that enables data sovereignty while maintaining Microsoft's ability to manage subscription licensing.

14. Conclusion

Azure AI Document Intelligence represents a transformative capability for any organisation that deals with documents at scale — which is virtually every organisation. By combining pre-built models for common document types with the ability to train custom models on your own data, and by providing a feedback loop mechanism for continuous improvement, it covers the full spectrum of real-world document processing needs.

The three capabilities covered in this chapter address distinct deployment scenarios:

- Pre-built models — for immediate, zero-configuration extraction on standard document types. Get running in hours, not weeks.
- Custom models with human feedback loops — for proprietary documents requiring high accuracy. Build a model that continuously learns from your team's domain expertise.
- On-premises Docker deployment — for regulated environments requiring complete data sovereignty, with the same API interface as the cloud service.

The Jupyter notebook workflow demonstrated in this chapter — from environment setup through authentication configuration, model training, human feedback collection, retraining, and validation — is a production-ready template for any document processing project. The versioned model management, the selective credential chain, and the feedback-to-JSON serialisation pipeline are patterns you can apply directly to your own domain-specific document extraction challenges.

As you build your GenAI and data science portfolio, document intelligence is one of the highest-value capabilities you can offer to enterprise clients. The ability to automate the extraction of structured data from unstructured documents — invoices, contracts, forms, records — and to continuously improve that automation through human feedback represents thousands of hours of manual work saved, reduced error rates, and fundamentally faster business processes.